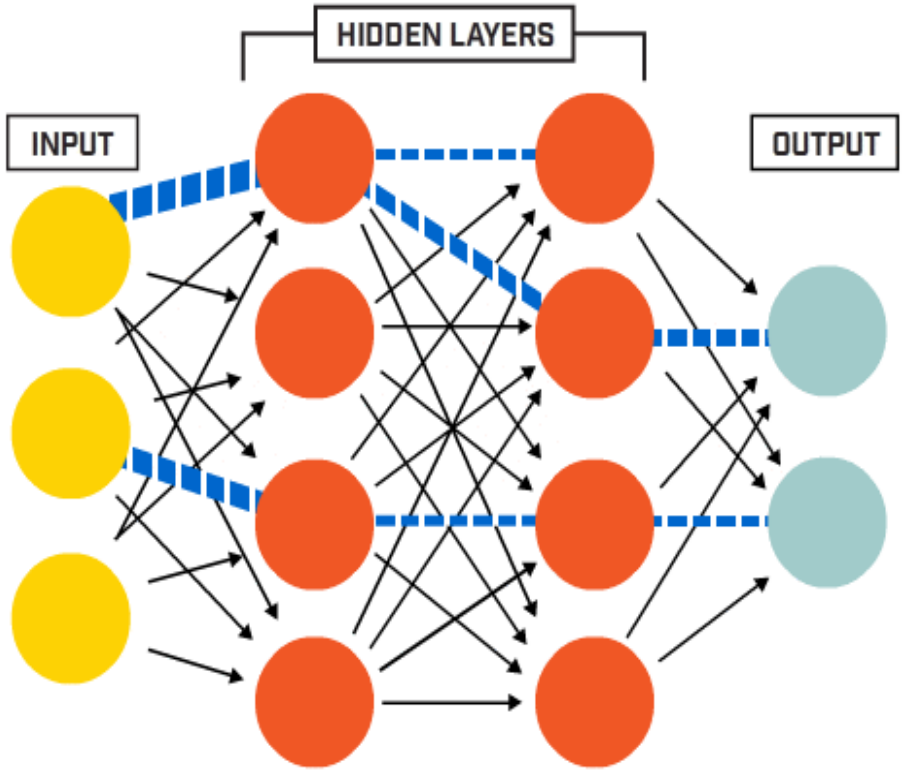
RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

ARTIFICIAL NEURAL NETWORK AND DEEP LEARNING



Dr. Viswavardhan Reddy Karna
Dept of AIML
RVCE

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

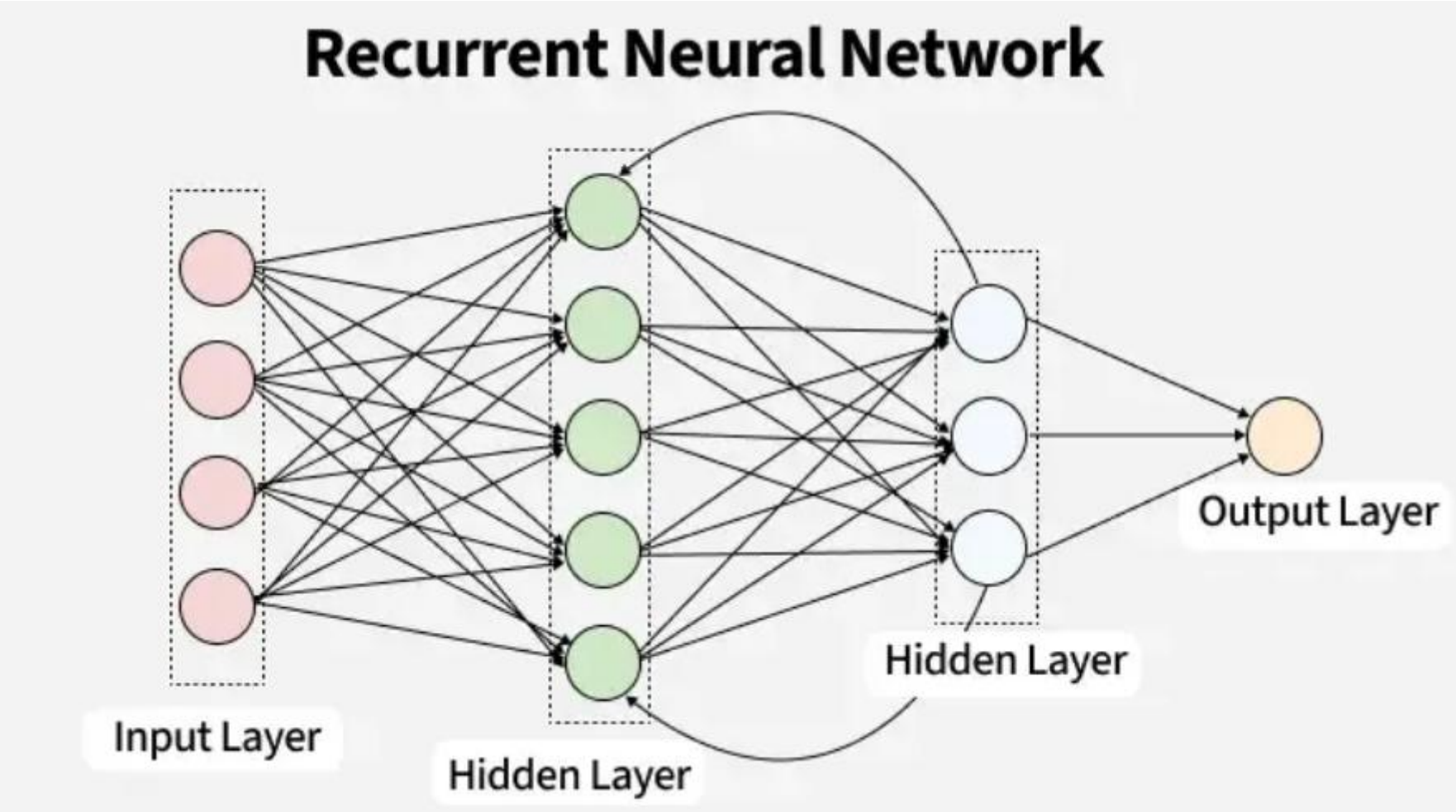
Course Contents

Unit –III	9Hrs.
Recurrent Neural Networks: Introduction and expressiveness of RNN. **Basic Structure of a RNN:** Language Modeling Example of RNN, Generating a Language Sample, Back propagation Through Time, Bidirectional Recurrent Networks, Multilayer Recurrent Networks. **Echo-State Networks, Long Short-Term Memory (LSTM), Gated Recurrent Units (GRUs)** **Applications of Recurrent Neural Networks:** Automatic Image Captioning, Sequence-to-Sequence Learning and Machine Translation, Sentence-Level Classification, Token-Level Classification with Linguistic Features, Time-Series Forecasting and Prediction, Temporal Recommender Systems, Secondary Protein Structure Prediction, End-to-End Speech Recognition, Handwriting Recognition	

Unit –IV	9Hrs.
Deep Reinforcement Learning: Introduction, **Stateless Algorithms:** Multi-Armed Bandits, Na¨ıve Algorithm, Greedy algorithm, Upper Bounding Methods **The Basic Framework of Reinforcement Learning:** Challenges of Reinforcement Learning, Simple Reinforcement Learning for Tic-Tac-Toe, ole of Deep Learning and a Straw-Man Algorithm. **Bootstrapping for Value Function Learning:** Deep Learning Models as Function Approximators, Example: Neural Network for Atari Setting, On-Policy Versus Off-Policy Methods: SARSA, Modeling States Versus State-Action Pairs, **Monte Carlo Tree Search** **Case Studies:** Alpha Go: Championship Level Play at Go, Alpha Zero: Enhancements to Zero Human Knowledge, Self-Learning Robots: Deep Learning of Locomotion Skills, Deep Learning of Visuomotor Skills, Building Conversational Systems: Deep Learning for Chat-Bots, Self-Driving Cars	

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Course Contents



Why RNNs? (Motivation)

- Traditional feedforward networks cannot model:
  - Sequential patterns – Ordered Relationship, one after the other
  - Time-series dependencies – Current value depending on the past
  - Language structure – Rules and patterns – on how words and sentences are formed

- Many real-world sequences:
  - Speech, text, DNA sequences
  - Machine sensor data, video frames

Introduction and expressiveness of RNN.

- A **Recurrent Neural Network (RNN)** is a type of neural network designed for **processing sequential data**.

- Unlike traditional feedforward networks, an RNN contains **loops** that allow information from **previous time steps** to persist and influence the current output.

How RNNs Work: RNNs process a sequence **step-by-step**:

- At each step t, they take the current input x_t
- Update the **hidden state** h_t using the previous state h_{t-1}
- Produce the output y_t

Introduction and expressiveness of RNN.

- This hidden state acts as the network's **memory**.

- Mathematical form:

$$h_t = f(Wx_t + Uh_{t-1} + b)$$

Key Features

- Handles **variable-length sequences:** **No fixed size of inputs, works with audio, sensor data streams and time series.**

- Retains **memory** of previous inputs – **hidden state captures the important information of previous steps**

- Suitable for sequential prediction- **speech recog, time series forecasting, video frame analysis**

Expressiveness of RNN.

- RNNs are **Turing complete** (Siegelmann & Sontag)

Can model any computable function with:

- Sufficient hidden units
- Proper training

Capable of learning:

- Context - previous words, events, or data points that help understand the current one.

- Long-range dependencies – long sentences, events over the time

- Grammar-like structures

Basic RNN Model

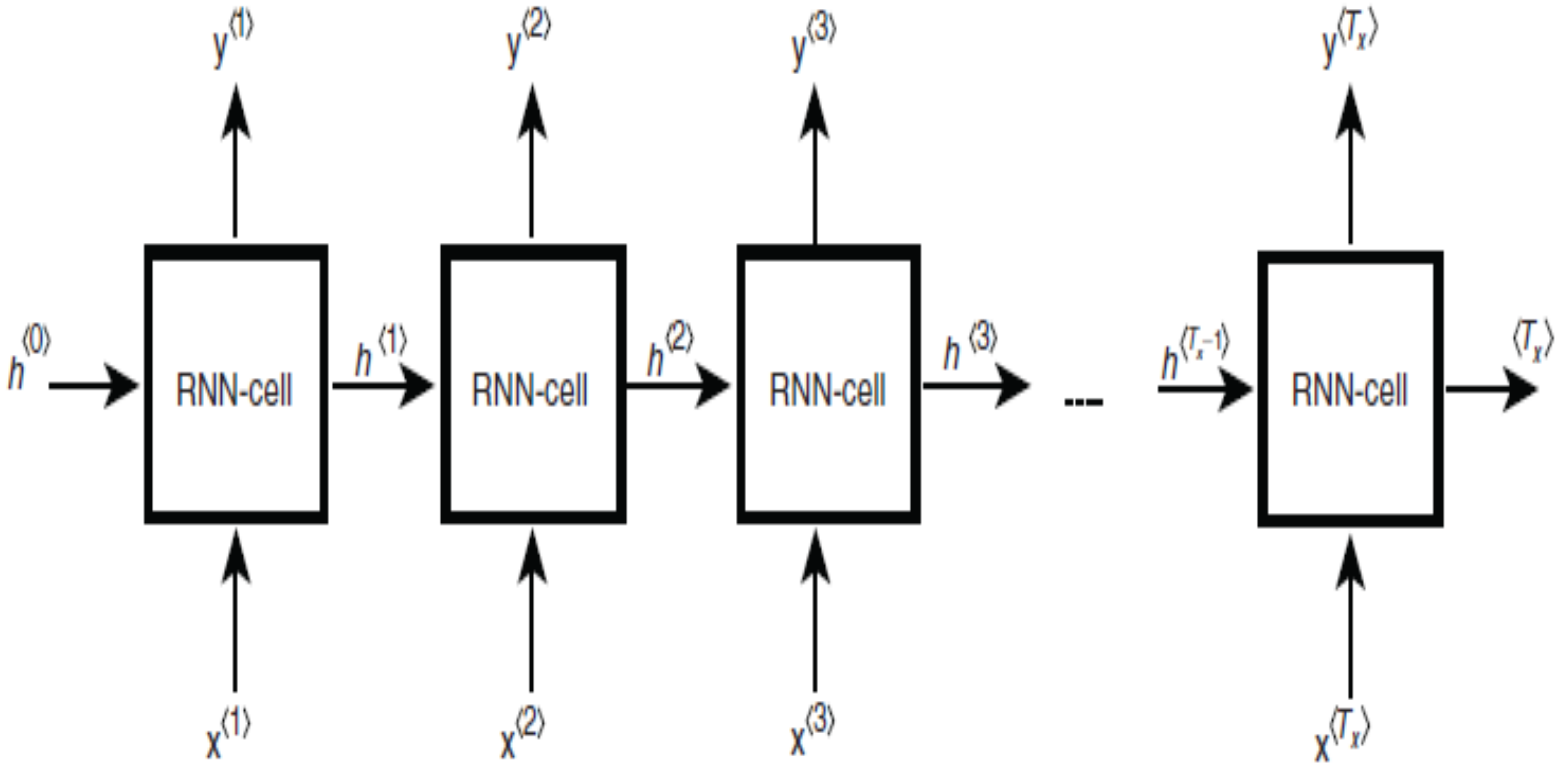


Fig. Basic RNN Model

Structure of RNN.

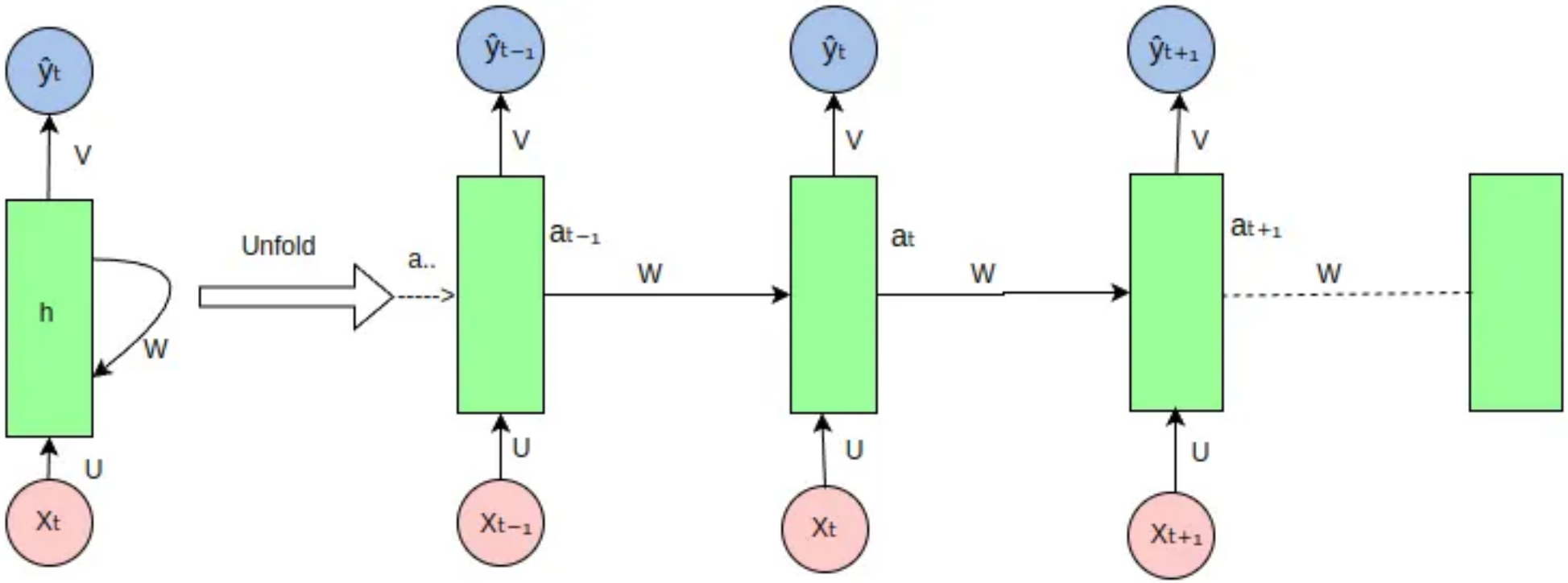


fig 3: RNN Unfolded

$$a_t = f(U * X_t + W * a_{t-1} + b)$$

Single RNN Cell

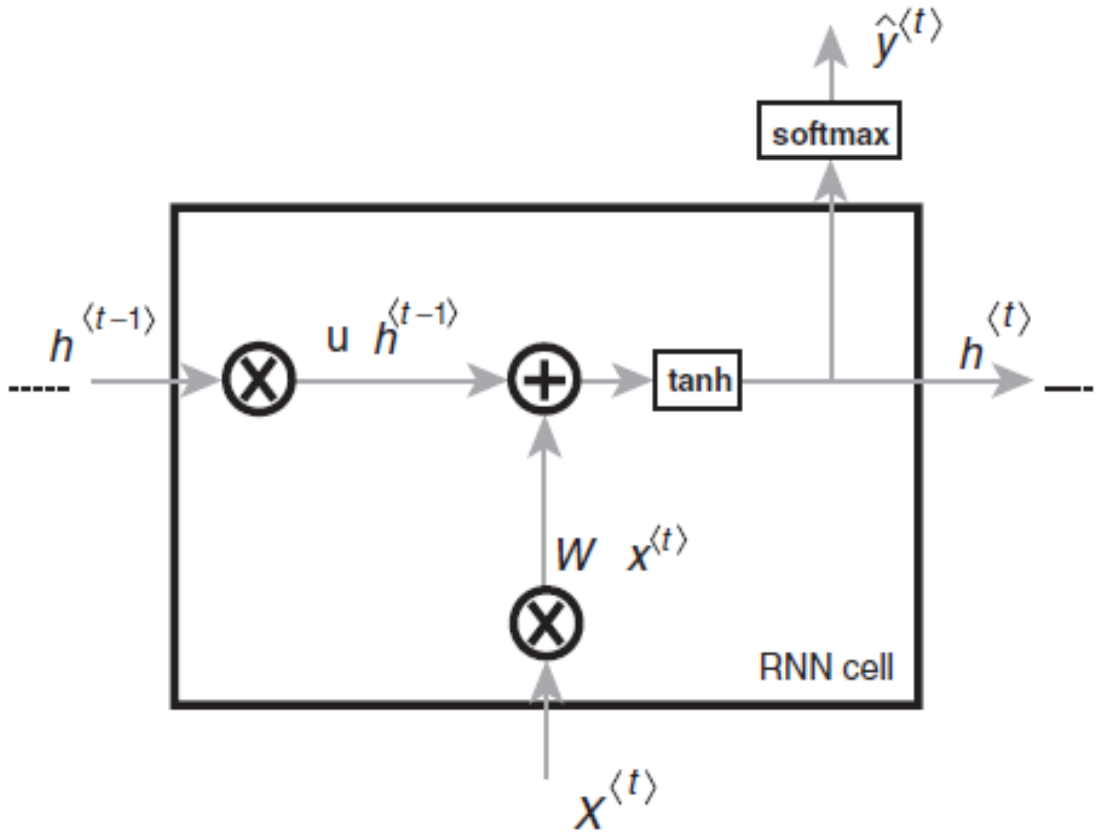


Fig. Single time step of an RNN cell

Structure of RNN.

CNN	RNN
It is suitable for spatial data such as images.	RNN is suitable for temporal data, also called sequential data.
CNN is considered to be more powerful than RNN.	RNN includes less feature compatibility when compared to CNN.
This network takes fixed size inputs and generates fixed size outputs.	RNN can handle arbitrary input/output lengths.
CNN is a type of feed-forward artificial neural network with variations of multilayer perceptrons designed to use minimal amounts of preprocessing.	RNN unlike feed forward neural networks - can use their internal memory to process arbitrary sequences of inputs.
CNNs use connectivity pattern between the neurons. This is inspired by the organization of the animal visual cortex, whose individual neurons are arranged in such a way that they respond to overlapping regions tiling the visual field.	Recurrent neural networks use time-series information - what a user spoke last will impact what he/she will speak next.
CNNs are ideal for images and video processing.	RNNs are ideal for text and speech analysis.

Structure of RNN.

Language Modeling Example

Task: Predict the next word or character given previous ones.

Example:

Input: "the cat sat on the"

Output: "mat“

the → [0.12, 0.55, -0.20, ...]
cat → [0.88, -0.11, 0.07, ...]
sat → [...]

"the" → RNN → h1
"cat" → RNN → h2
"sat" → RNN → h3
"on" → RNN → h4
"the" → RNN → h5

Steps:

1.Convert words → embeddings (dense vector that captures meaning)
2.Feed into the RNN one word at a time
3.Output probability distribution using softmax
4.Train via cross-entropy loss

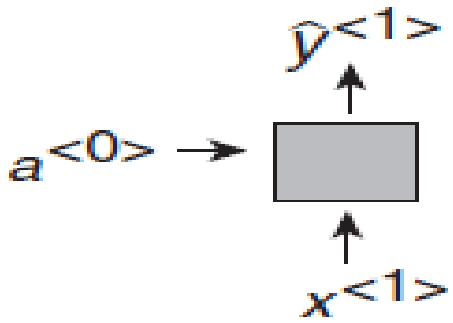
Word	Probability
mat	0.72
floor	0.10
bed	0.05
dog	0.01
...	...

RNN Topology

- One-to-One
- One-to-many
- Many-to-one
- Many-to-many for sequence input and sequence output
- Many-to-many for synced sequence input and output

One-to-One

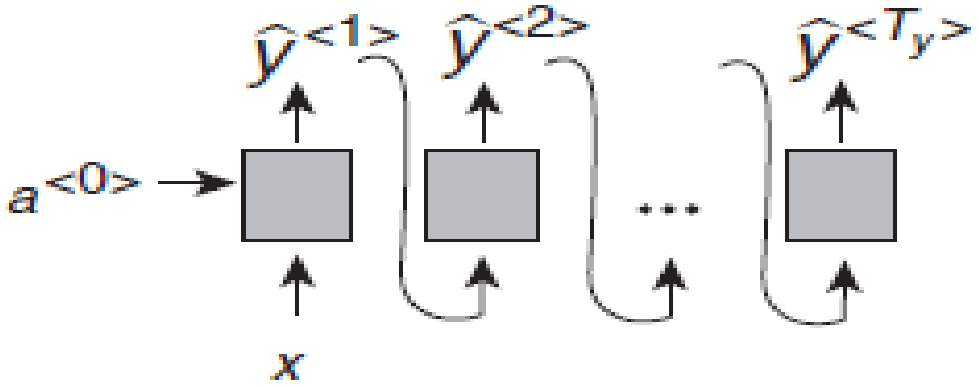
- **One-to-One** represents vanilla neural network (CNN) of processing without recurrent nets, from constant-sized input to constant-sized output (e.g., image classification).



One to one

One-to-Many

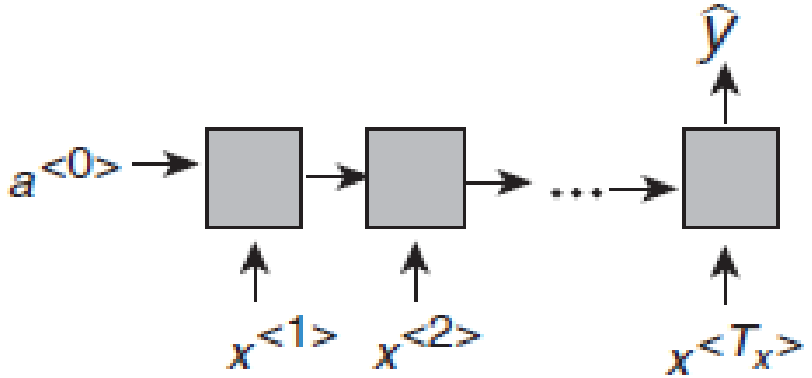
- A **one-to-many** neural network architecture refers to models where a **single, fixed-size input** is used to generate a **sequence of outputs**. (e.g., image captioning acquires an image as input and outputs a sentence of words).



One to many

Many-to-One

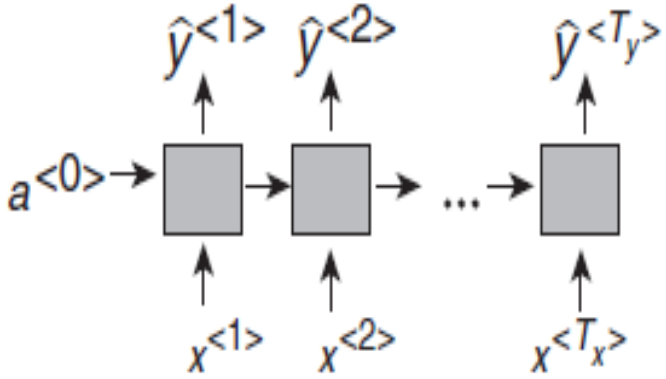
- A **many-to-one** neural network architecture processes a **sequence of inputs** and produces a **single output**.
- (e.g., sentiment analysis where a known sentence is classified as stating positive or negative sentiment).



Many to one

Many-to-Many

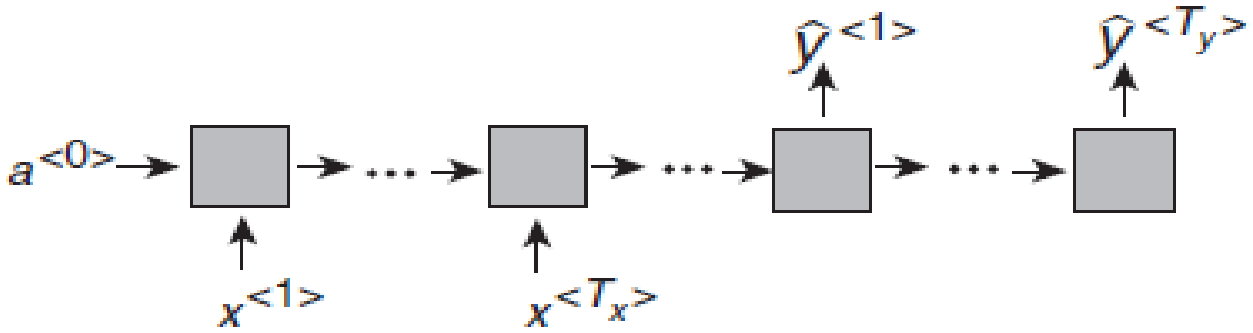
- A **many-to-many** neural network architecture handles **sequence inputs and sequence outputs**.
- A common example is **language translation**, where an RNN-based model (or modern Transformer) reads a sentence in English word-by-word and then generates a corresponding sentence in French word-by-word.
- Both the input and output are variable-length sequences.



Many to many

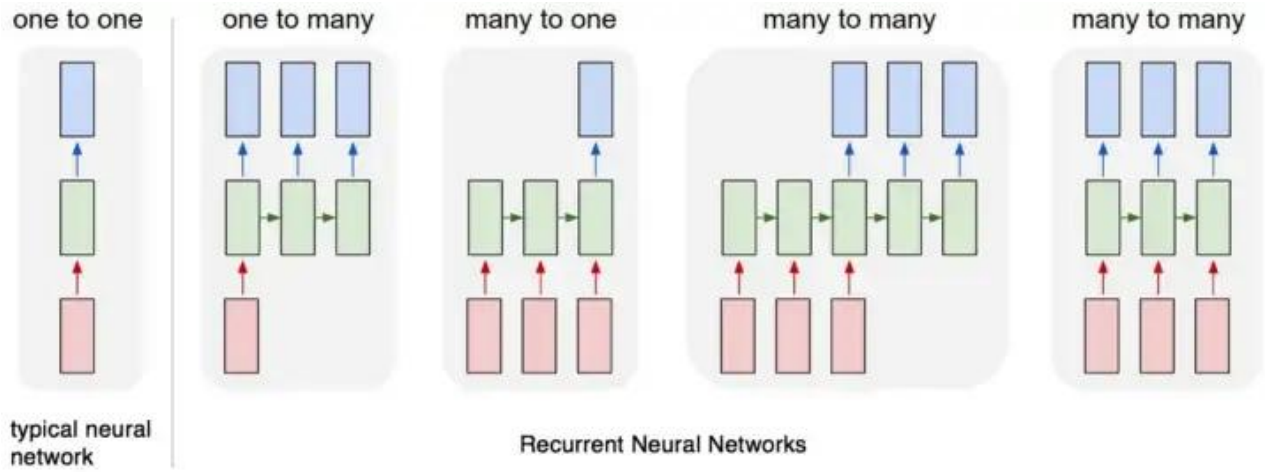
Many-to-Many (Contd…)

- **Many-to-many for synced sequence input and output, e.g., video classification,** where we want to label every frame.



Many to many

Summary of RNN topologies

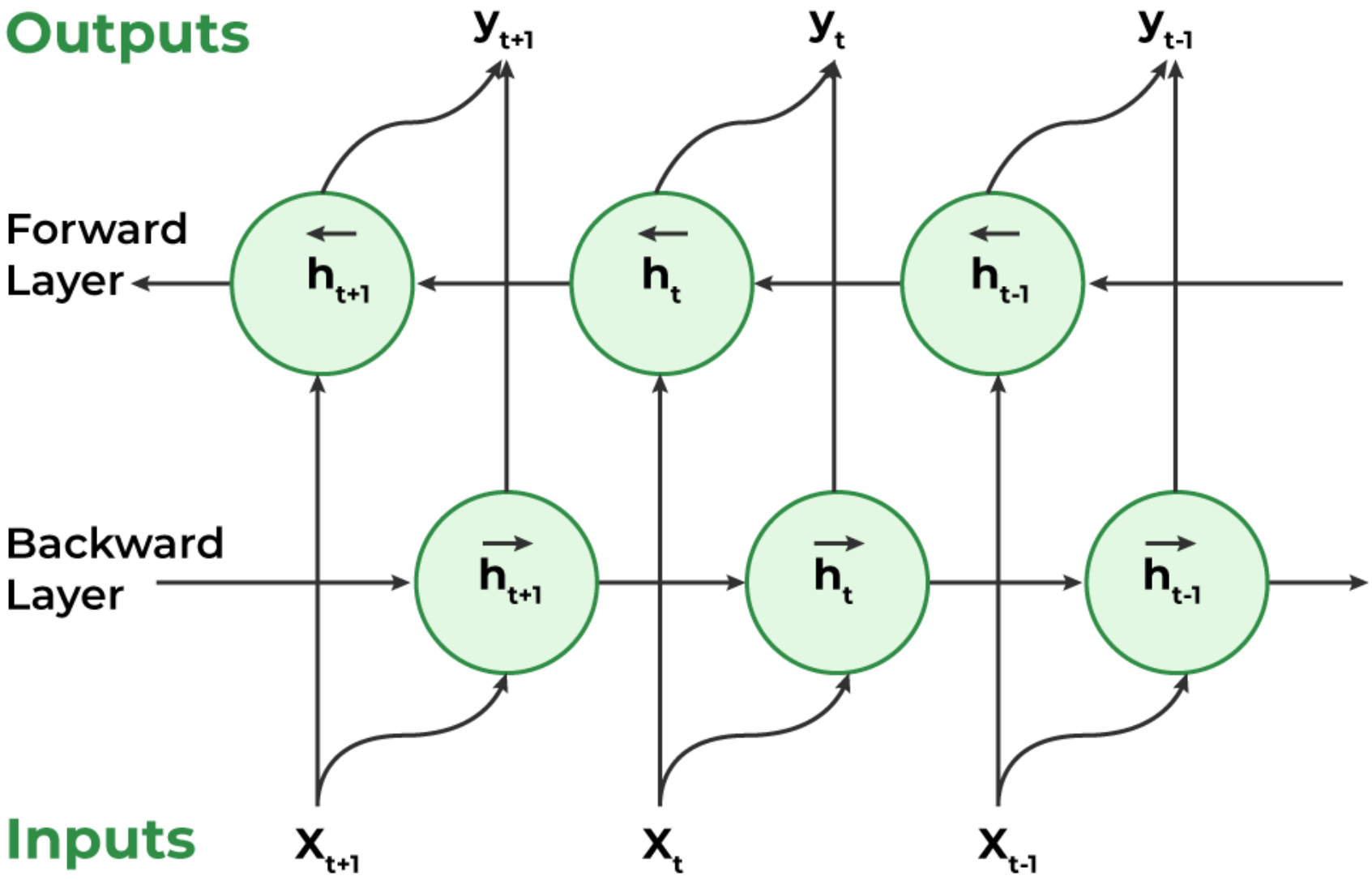


- From left to right: (1) Vanilla mode of processing without RNN, from fixed-sized input to fixed-sized output (e.g. image classification).
- (2) Sequence output (e.g. image captioning takes an image and outputs a sentence of words).
- (3) Sequence input (e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment).
- (4) Sequence input and sequence output (e.g. Machine Translation: an RNN reads a sentence in English and then outputs a sentence in French).
- (5) Synced sequence input and output (e.g. video classification where we wish to label each frame of the video).
- Notice that in every case are no pre-specified constraints on the lengths sequences because the recurrent transformation (green) is fixed and can be applied as many times as we like.

Bidirectional Recurrent Neural Network

- A Bidirectional Recurrent Neural Network (BRNN) is an extension of the traditional RNN that processes sequential data in both forward and backward directions.
- utilize both past and future context when making predictions
- Consider the sentence: *"I like apple. It is very healthy.“*
- "apple" refers to the fruit or the company based on the first sentence. (Forward)
- "apple" refers to the fruit, thanks to the future context provided by the second sentence ("It is very healthy.").

Bidirectional Recurrent Neural Network



Bidirectional Recurrent Neural Network

Advantages of BRNNs

- ✓ **Enhanced Context Understanding**
- ✓ **Improved Accuracy**
- ✓ **Better Handling of Variable-Length Sequences**
- ✓ **Increased Robustness**

Bidirectional Recurrent Neural Network

Challenges of BRNNs

- **High Computational Cost**
- **Longer Training Time**
- **Limited Real-Time Applicability**
- **Less Interpretability**

Bidirectional Recurrent Neural Network

Applications of Bidirectional Recurrent Neural Networks (BRNNs)

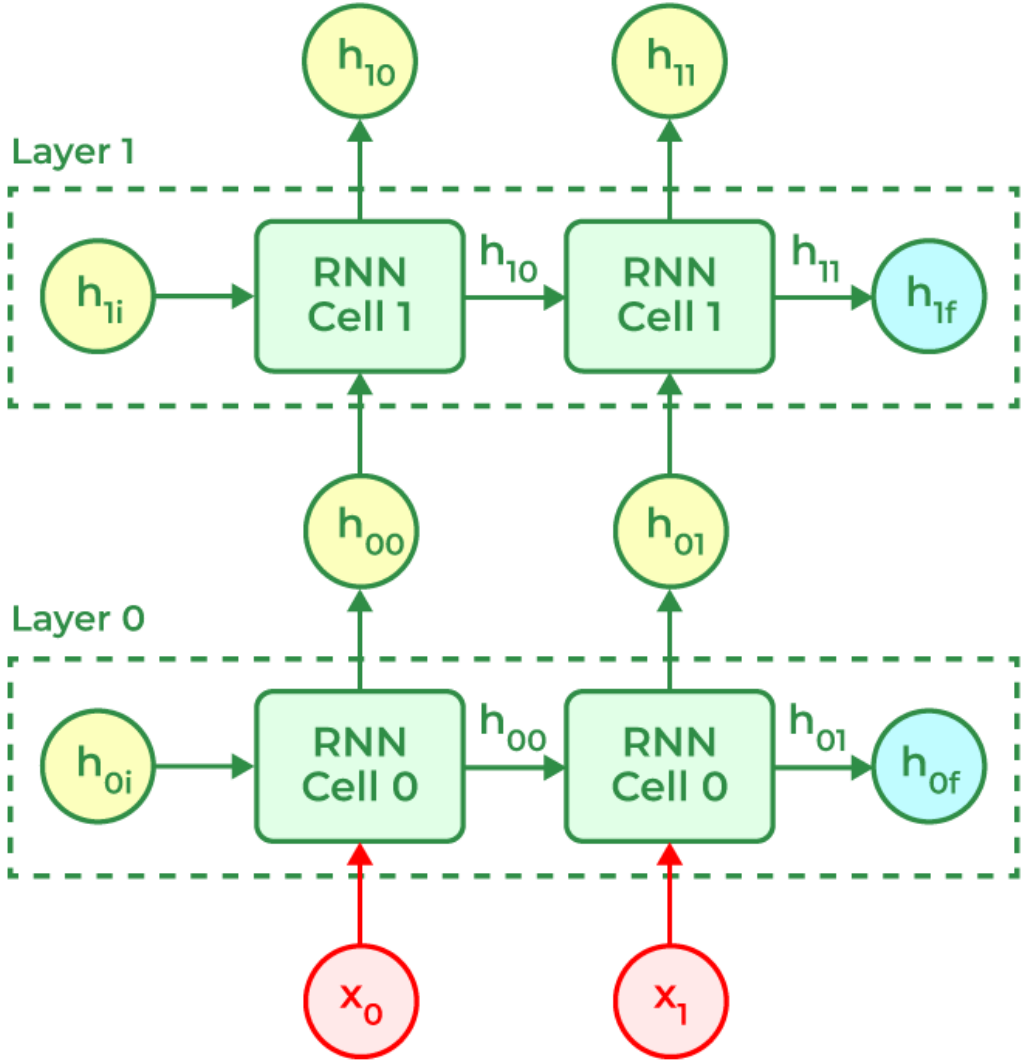
- **Sentiment Analysis:** *"The movie is not bad at all"*.
- **Speech Recognition**
- **Named Entity Recognition (NER):** Apple launched the new product in California." Apple – Not fruit, is a company, location is California.
- **Machine Translation**

Multilayer / Stacked RNN

- When an Input is passed to Layer 1:
  - The input (x_t (passes through the RNN layer 1. There, the hidden state gets updated as:

    $$h_t = \sigma(w_x x_t + w_h h_{t-1} + b_h)$$

  - where
    - h_t =present Hidden state
    - h_{t-1} =previous hidden state
    - x_t =input to the RNN layer
    - w_x =weights associated with the input
    - w_h =weights associated with the hidden layer
    - b_h =bias associated with RNN layer
    - σ= activation function



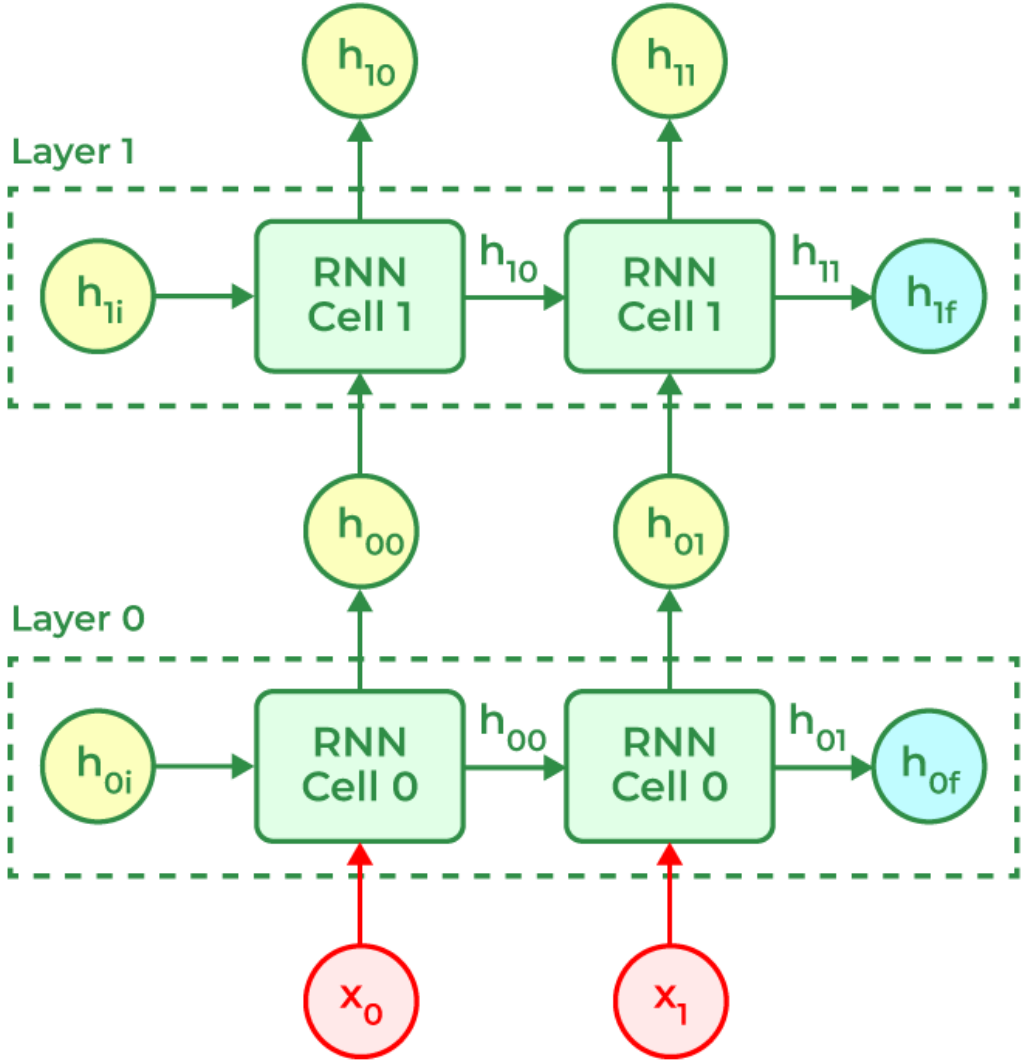
Multilayer / Stacked RNN

- The present hidden state is used in getting the output of the hidden layer, using an appropriate activation function.

- $y = Wh_t + b_y$where,

- W = Weights assigned to the layer
- h_t =hidden state
- b_y =bias associated with output layer

- For the second layer, the output of first RNN layer is fed into it, which goes through the same process again.



Echo-State Networks (ESNs)

- A specific kind of recurrent neural network (RNN) designed to handle sequential data efficiently.
- ESNs are created as a reservoir computing framework that includes a fixed, randomly initialized recurrent layer, known as the "reservoir."
- The key feature - echo-like dynamics of the reservoir, enabling them to capture and replicate temporal patterns in sequential input effectively.
- The way ESNs work is by linearly combining the input and reservoir states to generate the network's output. What sets them apart is that only the output layer is trained, while the reservoir weights remain fixed.
- This unique approach makes ESNs particularly useful in tasks where capturing temporal dependencies is critical, such as time-series prediction and signal processing.

Echo-State Networks (ESNs)

What is Echo State Networks?

- Imagine it as a three-part orchestra: there's the input layer, a reservoir filled with randomly initialized interconnected neurons, and the output layer.

- **Reservoir:** A pool of interconnected neurons works together to remember patterns in the data. It's like having a memory of what happened before.

- **Training:** We show the ESN some of our data and let it learn. It doesn't learn everything but gets the hang of the patterns. It's like giving a few examples to a friend so they can understand how to predict.

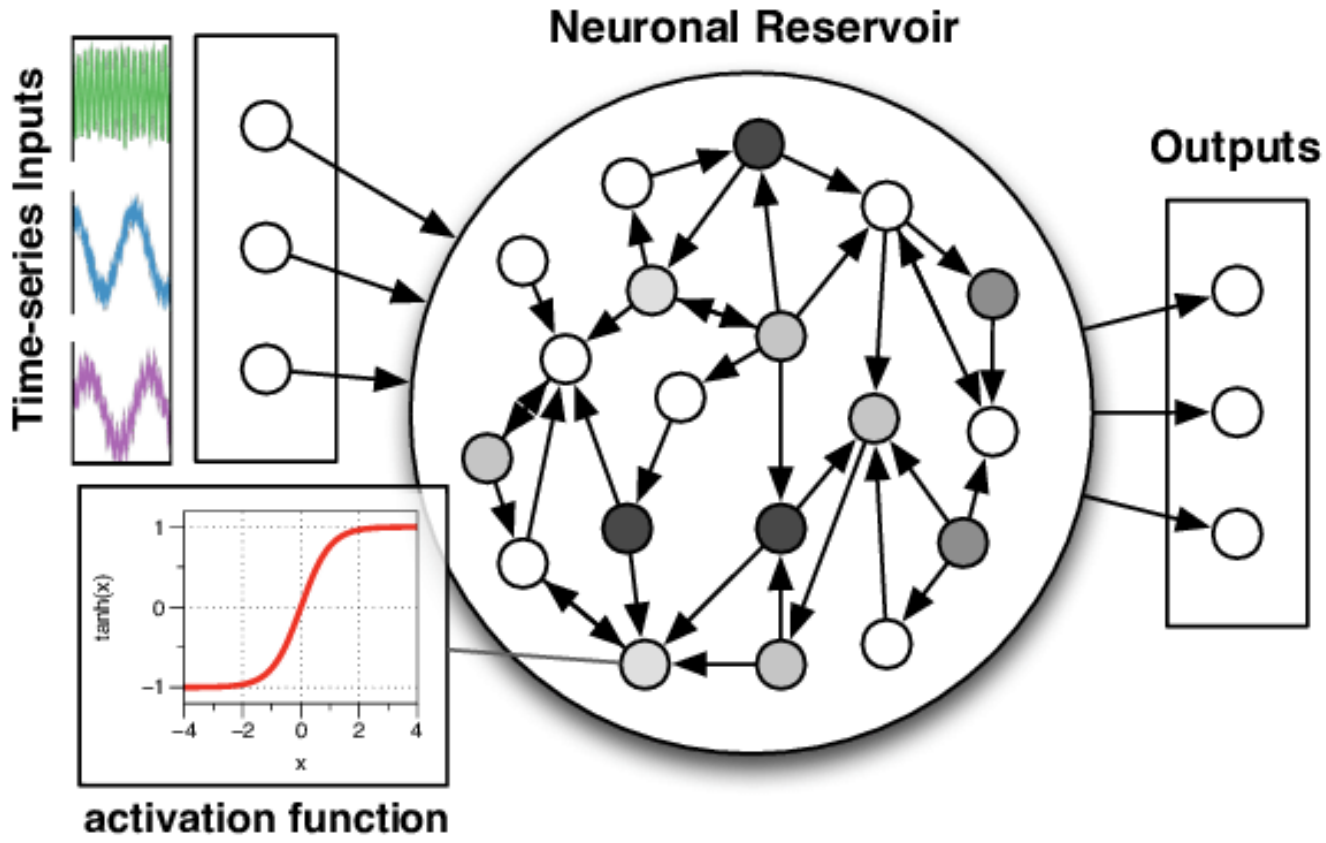
- **Predicting:** when we give the ESN a new piece of data (like the past temperatures), it uses what it learned to guess what comes next.

- **Output:** The ESN gives us its prediction, and we can compare it to the real answer. If it's good, great! If not, we might need to tweak things a bit.

Echo-State Networks (ESNs)

What is Echo State Networks?

- Imagine it as a three-part orchestra: there's the input layer, a reservoir filled with randomly initialized interconnected neurons, and the output layer.



Echo-State Networks (ESNs)

Components

u(t) → Input vector, **x(t)** → Reservoir state vector (high-dimensional), **y(t)** → Output vector, **W_in** → Input-to-reservoir weights, **W** → Recurrent reservoir weights (fixed), **W_out** → Trainable output weights

State Update Equation

The reservoir state is updated as: $x(t) = f\left(W_{in}u(t) + Wx(t-1)\right)$

Where $f(\cdot)$ is typically **tanh** or **ReLU**

The output is: $y(t) = W_{out}\, x(t)$

Training Only W_{out} is trained using **ridge regression**:

$$W_{out} = YX^T\left(XX^T + \lambda I\right)^{-1}$$

Where: X = matrix of reservoir states

Y = target outputs

λ = regularization parameter

Echo-State Networks (ESNs)

Motivation

Traditional RNN training suffers from:

- Vanishing/exploding gradients
- High training cost due to recurrent weight updates

ESNs solve this by:

- Using a **large random recurrent reservoir**
- Only training a **linear readout layer**

Echo-State Networks (ESNs)

Advantages

- Very fast training (only linear layer trained)
- Handles long-term dependencies better than vanilla RNNs
- Good for time-series forecasting

Limitations

- Selecting spectral radius and reservoir size is tricky
- Random reservoir may not optimally capture dynamics

Applications

- Speech recognition
- System modelling
- Time-series forecasting (weather, stock prediction)
- Biological signal processing (EEG, ECG)

Problem with Long-Term Dependencies in RNN

- Recurrent Neural Networks (RNNs) - handle sequential data by maintaining a hidden state capturing information from previous time steps.

- However, they often face challenges in learning long-term dependencies where information from distant time steps becomes crucial for making accurate predictions for current state.

- This problem is known as the vanishing gradient or exploding gradient problem.

Vanishing Gradient: When training a model over time, the gradients which help the model learn can shrink as they pass through many steps.
This makes it hard for the model to learn long-term patterns since earlier information becomes almost irrelevant.
Exploding Gradient: Sometimes gradients can grow too large causing instability. This makes it difficult for the model to learn properly as the updates to the model become erratic and unpredictable.

Long Short-Term Memory (LSTM)

Introduction

LSTMs overcome the **vanishing gradient problem** by introducing **memory cells** and **gates** to control information flow.

They are effective for:

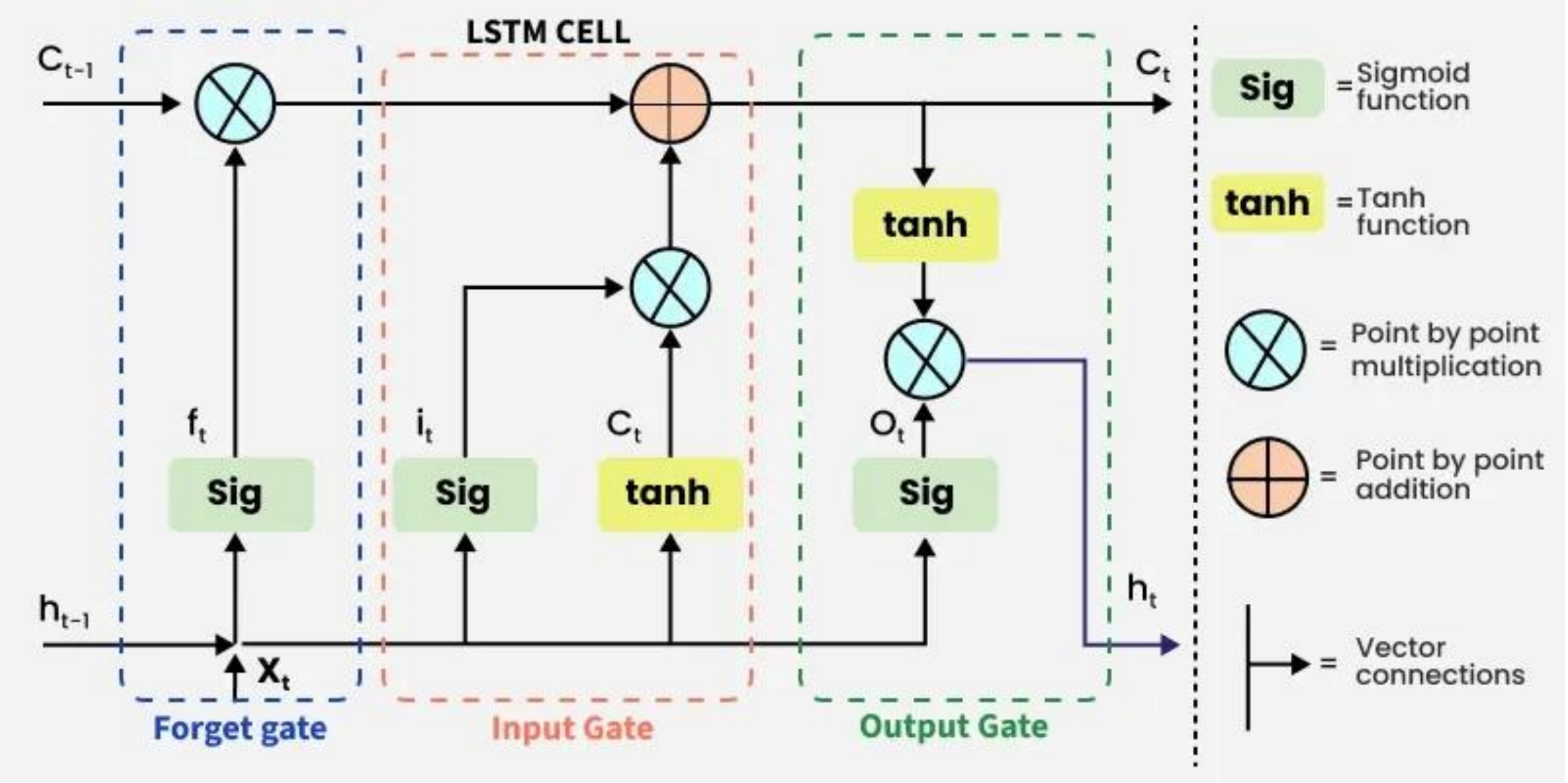
- Sequential data
- Long-term dependency learning

LSTM Architecture

An LSTM cell contains:

- Forget Gate
- Input Gate
- Candidate Memory
- Output Gate

Long Short-Term Memory (LSTM)



Long Short-Term Memory (LSTM)

- The information that is no longer useful in the cell state is removed with the forget gate.
- Two inputs (x_t)input at the particular time and (h_{t-1})previous cell output are fed to the gate and multiplied with weight matrices followed by the addition of bias.
- The resultant is passed through sigmoid activation function which gives output in range of [0,1].
- If for a particular cell state the output is 0 or near to 0, the piece of information is forgotten and for output of 1 or near to 1, the information is retained for future use.
- The equation for the forget gate is:

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

Long Short-Term Memory (LSTM)

- **Input gate**
- The addition of useful information to the cell state is done by the input gate.

- First the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs h_{t-1} and x_t .

- Then, a vector is created using *tanh* function that gives an output from -1 to +1 which contains all the possible values from h_{t-1} and x_t .
- At last the values of the vector and the regulated values are multiplied to obtain the useful information. The equation for the input gate is:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\hat{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

- We multiply the previous state by f_t effectively filtering out the information we had decided to ignore earlier.
- Then we add $i_t \odot C_t$ which represents the new candidate values scaled by how much we decided to update each state value.

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$$

where

- $\odot$ denotes element-wise multiplication
- tanh is activation function

Long Short-Term Memory (LSTM)

Output gate

- The output gate is responsible for deciding what part of the current cell state should be sent as the hidden state (output) for this time step.

- First, the gate uses a sigmoid function to determine which information from the current cell state will be output.

- This is done using the previous hidden state h_{t-1} and the current input x_t:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- Next, the current cell state C_t is passed through a tanh activation to scale its values between $-1$and $+1$.

- Finally, this transformed cell state is multiplied element-wise with o_t to produce the hidden state h_t:

Long Short-Term Memory (LSTM)

Here:

o_t is the output gate activation.

C_t is the current cell state.

$\odot$represents element-wise multiplication.

$\sigma$is the sigmoid activation function.

This hidden state h_t is then passed to the next time step and can also be used for generating the output of the network.

Applications

Some of the famous applications of LSTM includes:

- Language Modelling
- Speech Recognition
- Time series forecasting,
- Anamoly detection, Recommender system,
- Video analysis

Gated Recurrent Unit Networks

- LSTMs are very complex structure with higher computational cost.

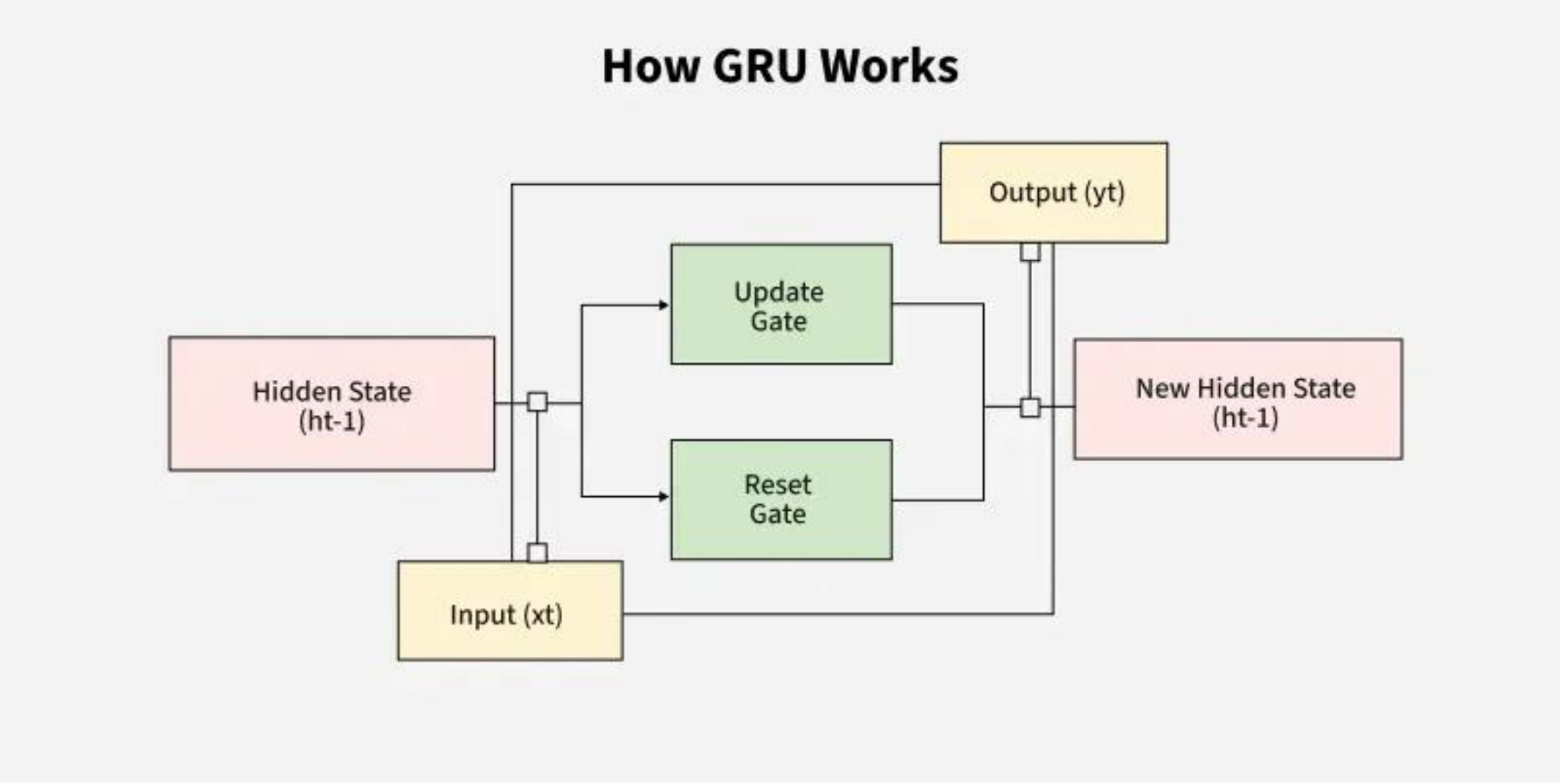
- To overcome this Gated Recurrent Unit (GRU) where introduced which uses LSTM architecture by merging its gating mechanisms offering a more efficient solution for many sequential tasks without sacrificing performance.

The GRU consists of two main gates:

- **Update Gate** (z_t this gate decides how much information from previous hidden state should be retained for the next time step.

- **Reset Gate** (r_t this gate determines how much of the past hidden state should be forgotten.

Gated Recurrent Unit Networks



Gated Recurrent Unit Networks

Equations for GRU Operations

The internal workings of a GRU can be described using following equations:

1. Reset gate:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

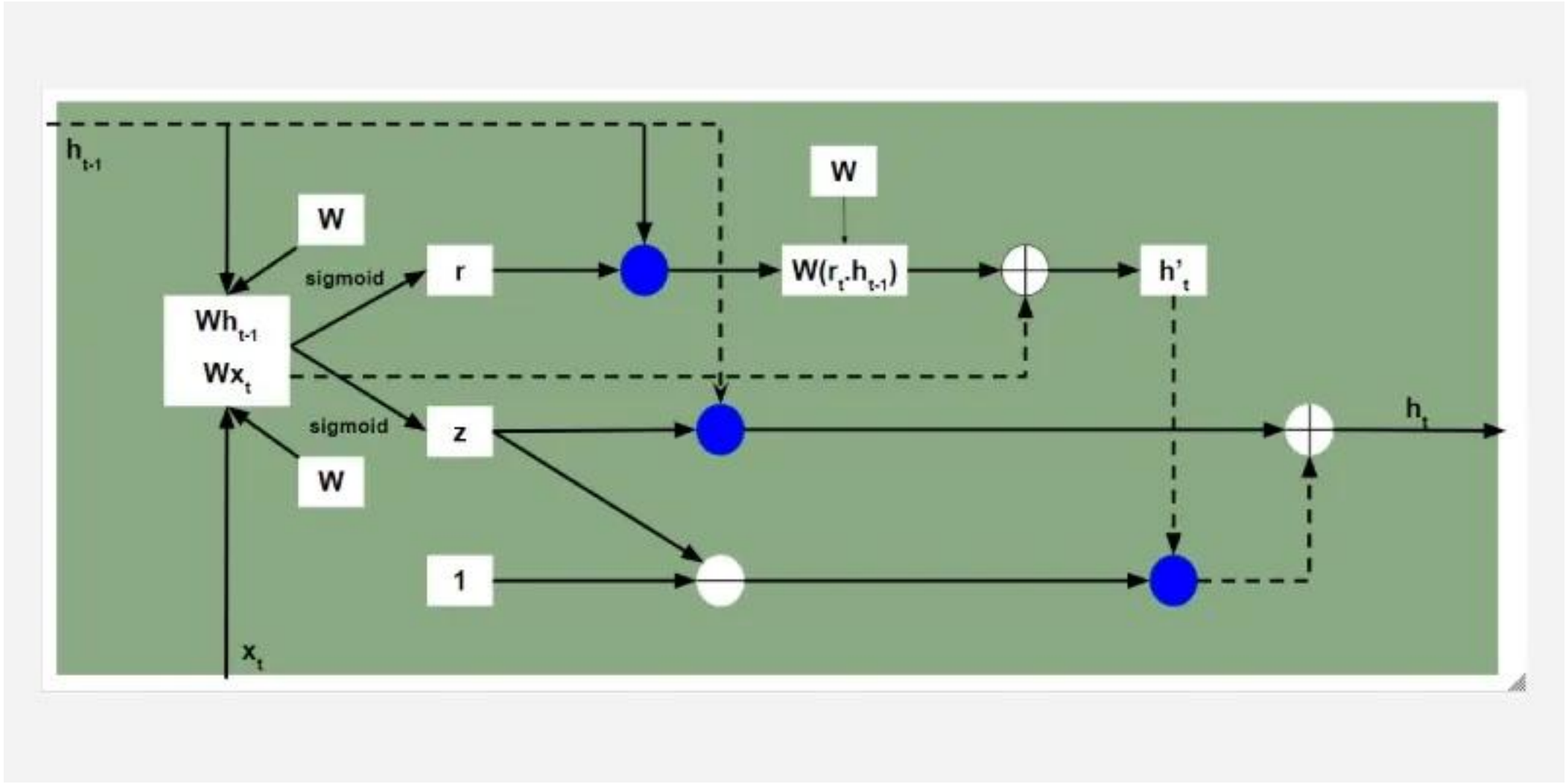
The reset gate determines how much of the previous hidden state h_{t-1} should be forgotten.

2. Update gate:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

The update gate controls how much of the new information x_t should be used to update the hidden state.

Gated Recurrent Unit Networks



Gated Recurrent Unit Networks

Candidate hidden state: $$h'_t = \tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t])$$

This is the potential new hidden state calculated based on the current input and the previous hidden state.

4. Hidden state:

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot h'_t$$

The final hidden state is a weighted average of the previous hidden state h_{t-1} and the candidate hidden state h'_t based on the update gate z_t.

Gated Recurrent Unit Networks

Feature	LSTM (Long Short-Term Memory)	GRU (Gated Recurrent Unit)
Gates	3 (Input, Forget, Output)	2 (Update, Reset)
Cell State	Yes it has cell state	No (Hidden state only)
Training Speed	Slower due to complexity	Faster due to simpler architecture
Computational Load	Higher due to more gates and parameters	Lower due to fewer gates and parameters
Performance	Often better in tasks requiring long-term memory	Performs similarly in many tasks with less complexity